



Estratégias de Diversificação em Meta heurísticas para Otimização em Engenharia Elétrica usando um Framework em Python

Pedro Victor Rodrigues Veras , Dieison Mariano Farias Rocha e
Rainer Zanghi

Introdução

Ferramentas de **otimização**, como as **meta-heurísticas**, desempenham um papel fundamental na busca por soluções eficientes para problemas complexos em diversas áreas da **engenharia elétrica**.

Este trabalho apresenta o desenvolvimento de um **framework** em código aberto para resolver problemas de otimização em **Engenharia Elétrica**, integrando técnicas de diversificação e bibliotecas de **Python** para **meta-heurísticas** e análise de **sistemas elétricos de potência**.





Objetivos da Pesquisa

1

Criar um Framework

Desenvolver um framework de software de otimização em código aberto para aplicações dos Sistemas Elétricos de Potência (SEP) em Python, integrando bibliotecas de meta-heurísticas e de análise de SEP.

2

Estratégia de Diversificação

Implementar a estratégia de diversificação **Repopulação com Conjunto Elite (RCE)** no framework.

3

Automação de Testes

Automatizar a execução de testes de benchmark para avaliar o desempenho do framework.



Otimização em Engenharia Elétrica

1

Otimização de Parâmetros de Estabilizadores de Sistemas de Potência:

Ajustar os parâmetros desses dispositivos para melhorar a estabilidade do sistema elétrico. Poderia ser usado um programa para encontrar a combinação ideal de parâmetros que maximizam a estabilidade.

2

Agendamento de Intervenções em Redes Elétricas:

Definir o melhor momento para realizar manutenções ou reparos na rede elétrica, minimizando o impacto no fornecimento de energia. Poderia ser usado para encontrar um cronograma de intervenções que minimize as interrupções no fornecimento.

3

Despacho Econômico:

Determinar a maneira mais econômica de operar um sistema de geração de energia, considerando os custos de produção, a demanda e as restrições do sistema.

Poderia encontrar a programação de usinas que atenda à demanda de energia a um custo mínimo, respeitando os limites de geração de cada usina.

Meta-heurísticas, Algoritmos Genéticos, Evolutivos e Diversificação - Juntando os conceitos

Meta-heurísticas

São estratégias inteligentes para encontrar soluções aproximadas, mas eficazes, para problemas complexos de otimização, dentro de um tempo computacional aceitável, mesmo que não garantam a melhor solução absoluta.

Algoritmos Evolutivos

Os Algoritmos Evolutivos (AEs) são uma classe de metaheurísticas inspiradas em processos naturais de evolução, como a **Seleção Natural**. Eles são amplamente utilizados para resolver problemas de otimização complexos.

Diversificação

A diversificação é crucial para evitar convergência prematura em metaheurísticas. Uma estratégia promissora é a **Repopulação com Conjunto Elite (RCE)**, podendo ser utilizada em problemas de agendamento em redes elétricas.



Bibliotecas de Código Aberto para Otimização na Linguagem Python

Flexibilidade e Modularidade

A biblioteca escolhida deve oferecer um ambiente flexível e modular, permitindo a aplicação do framework para diversos problemas de otimização e a inclusão de novas metaheurísticas.

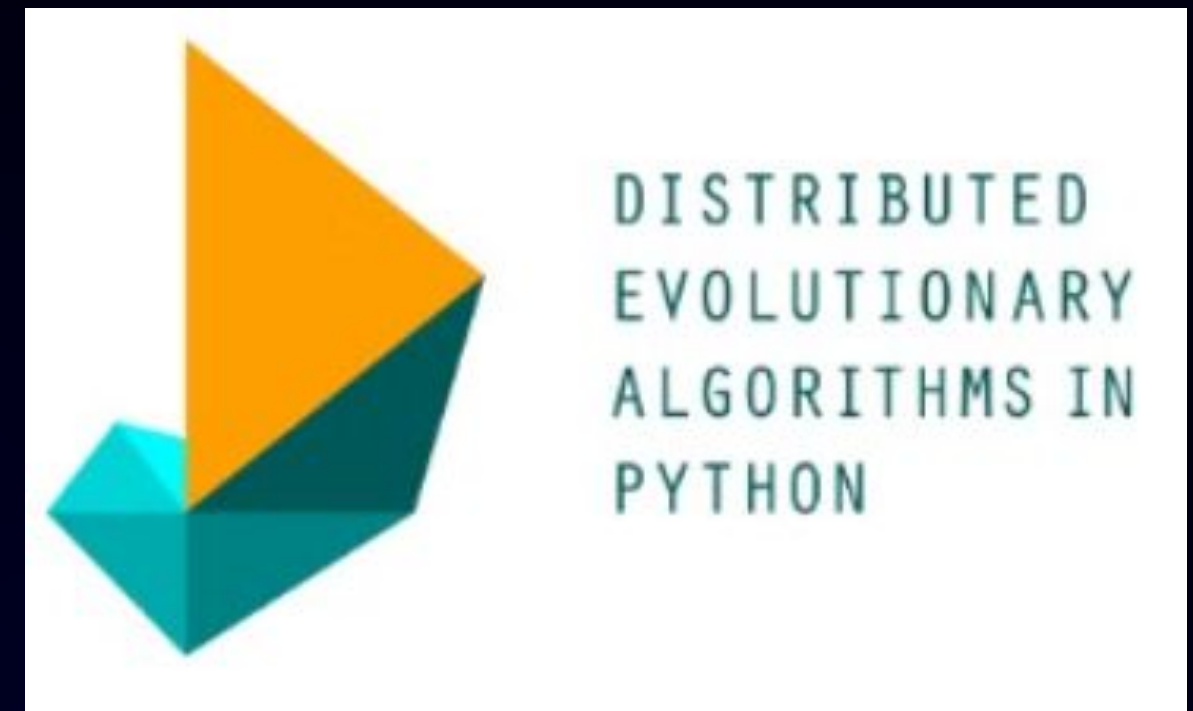
Bibliotecas Populares

Algumas das bibliotecas de código aberto em Python mais conhecidas para otimização usando metaheurísticas são o DEAP, o NiaPy e o Optuna.

A biblioteca escolhida foi a **DEAP** pela facilidade dos seus operadores genéticos.

Critérios de Escolha

Ao selecionar uma biblioteca de código aberto para implementar metaheurísticas, é importante considerar fatores como o tipo de licença, o número de metaheurísticas implementadas, o suporte a novos operadores e a existência de uma comunidade ativa.



Código Exemplo básico da documentação do DEAP

```
import random
from deap import creator, base, tools, algorithms

creator.create("FitnessMax", base.Fitness, weights=(1.0,))
creator.create("Individual", list, fitness=creator.FitnessMax)

toolbox = base.Toolbox()

toolbox.register("attr_bool", random.randint, 0, 1)
toolbox.register("individual", tools.initRepeat, creator.Individual, toolbox.attr_bool, n=100)
toolbox.register("population", tools.initRepeat, list, toolbox.individual)

def evalOneMax(individual):
    return sum(individual),

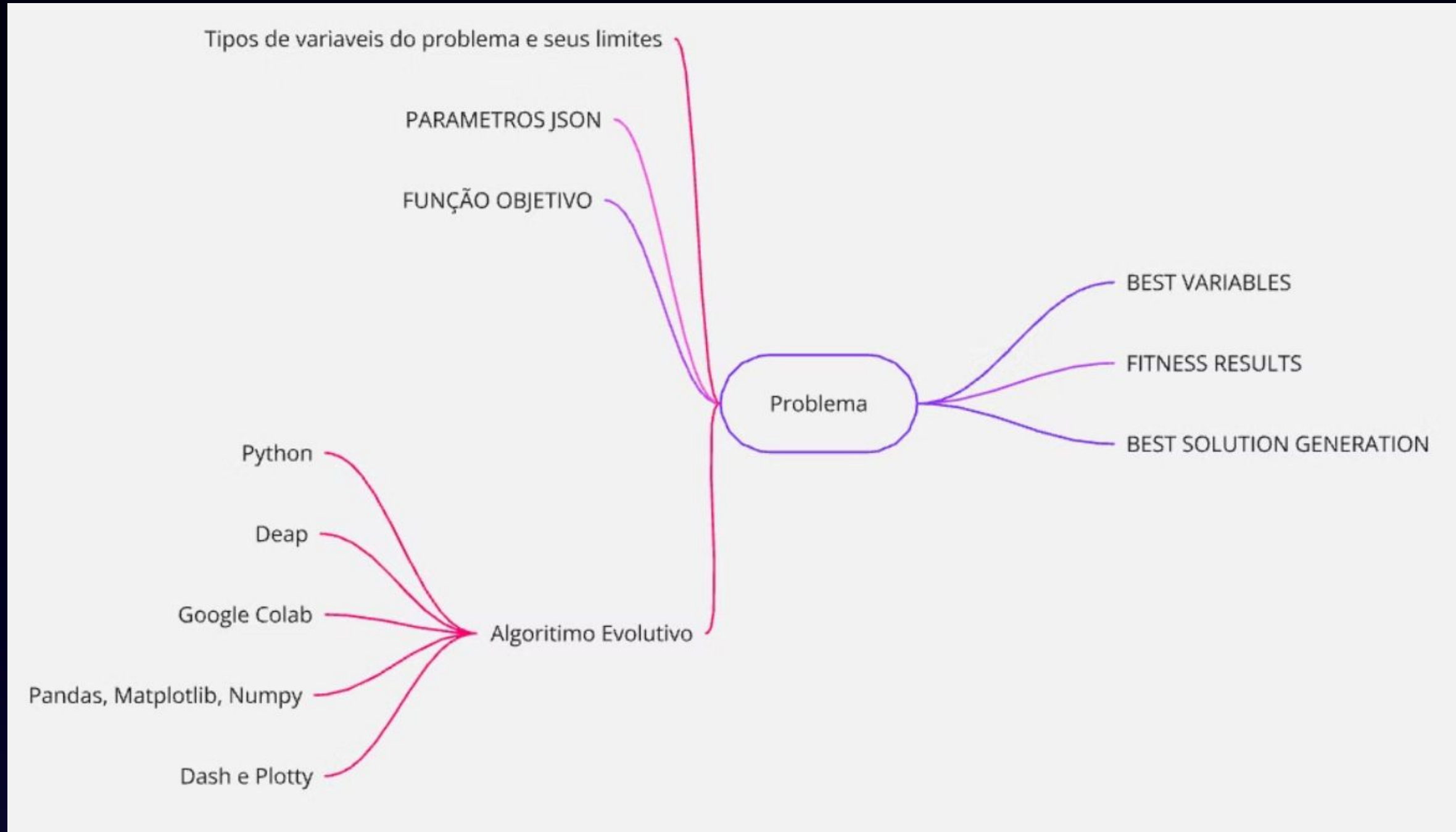
toolbox.register("evaluate", evalOneMax)
toolbox.register("mate", tools.cxTwoPoint)
toolbox.register("mutate", tools.mutFlipBit, indpb=0.05)
toolbox.register("select", tools.selTournament, tournsize=3)

population = toolbox.population(n=300)

NGEN=40
for gen in range(NGEN):
    offspring = algorithms.varAnd(population, toolbox, cxpb=0.5, mutpb=0.1)
    fits = toolbox.map(toolbox.evaluate, offspring)
    for fit, ind in zip(fits, offspring):
        ind.fitness.values = fit
    population = toolbox.select(offspring, k=len(population))

top10 = tools.selBest(population, k=10)
```

Mapa mental do projeto





Algoritmo Evolutivo

Foi implementado um algoritmo evolutivo baseado na biblioteca *DEAP* em linguagem *Python*, seguindo os passos principais:

- **Definição da Função Fitness**
- **Criação da População Inicial**
- **Seleção de Indivíduos**
- **Cruzamento**
- **Mutação**
- **Convergência**

Código aberto disponível

```
# Exemplo de individuo de tamanho 10
ind1 = [1,2,3,4,5,6,7,8,9,10]

# Função Objetivo
def evaluate(individual, decision_variables):
    a = sum(individual)
    b = len(individual)

    return a / b

# Funções Objetivo de benchmarking
def rastrigin_decisionVariables(individual, decision_variables):
    rastrigin = 10 * len(decision_variables)
    for i in range(len(decision_variables)):
        rastrigin += individual[i] * individual[i] - 10 * (
            math.cos(2 * np.pi * individual[i]))
    )
    return rastrigin
```

Código aberto disponível

```
if __name__ == "__main__":

    # Instanciando os Objetos
    setup = Setup(params)
    alg = AlgoritmoEvolutivoRCE(setup)
    data_visual = DataExploration()

    # Loop Algoritmo Evolutivo podendo receber a função objetivo e as variaveis do problema
    pop_with_repopulation, logbook_with_repopulation, best_variables = alg.run(
        RCE=True,
        fitness_function=rastrigin_decisionVariables,
        decision_variables=(ind1),
    )

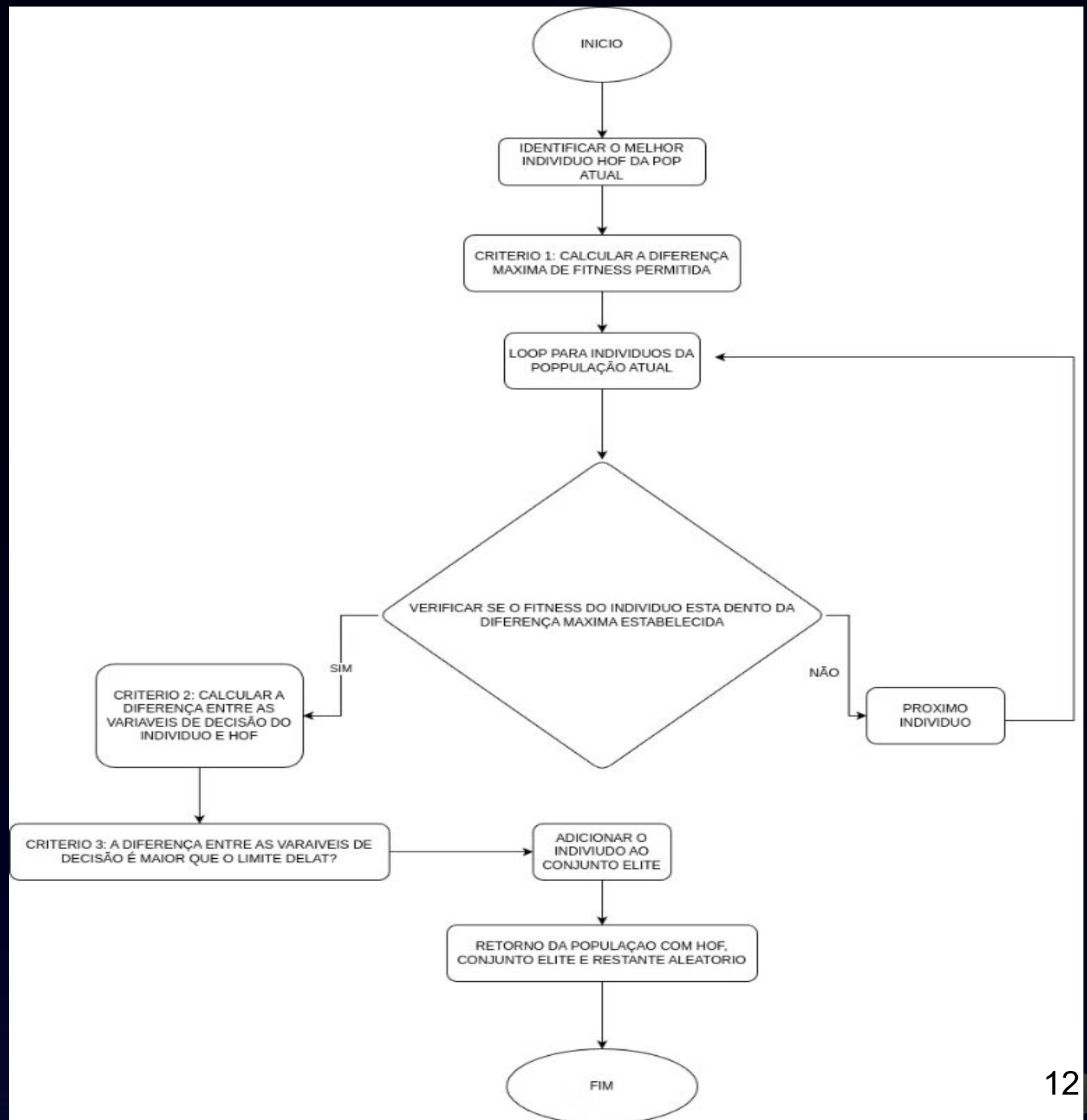
    print("\n\nEvolução concluída - 100%")

    # Grafico Benchmark
    data_visual.show_rastrigin_benchmark(logbook_with_repopulation, best_variables)

    # Resultados
    x, y, z = data_visual.visualize(
        logbook_with_repopulation, pop_with_repopulation, repopulation=True
    )
```


Estratégia Repopulação Conjunto Elite(RCE)

- **Melhoria na Qualidade das Soluções:** A presença de picos nos gráficos de aptidão média após a aplicação da RCE indica um aumento na diversidade da população e a exploração de novas áreas do espaço de busca, levando a soluções de maior qualidade.
- **Prevenção da Convergência Prematura:** A introdução de novos indivíduos aleatórios a cada ciclo de repopulação impede que o algoritmo convirja para um mínimo local, explorando o espaço de busca de forma mais abrangente.
- **Eficiência na Resolução de Problemas Complexos:** A combinação do conjunto elite com a população restante aleatória torna a RCE eficaz na resolução de problemas complexos com múltiplos mínimos locais, como a função Rastrigin, frequentemente encontrada em otimização de sistemas elétricos de potência.



Resultados e Discussões - Simulação e Testes

1

Automação das Simulações

O framework implementa um processo automatizado de execução do Algoritmo Evolutivo com a estratégia de Repopulação com Conjunto Elite (RCE). Cada conjunto de parâmetros é executado 20 vezes, e estatísticas descritivas são geradas para cada simulação.

2

Testes com Função de Benchmark

Para validar o desempenho do framework, foram realizados testes utilizando a função de benchmark Rastrigin, que é amplamente utilizada para avaliar a eficácia de algoritmos de otimização.

3

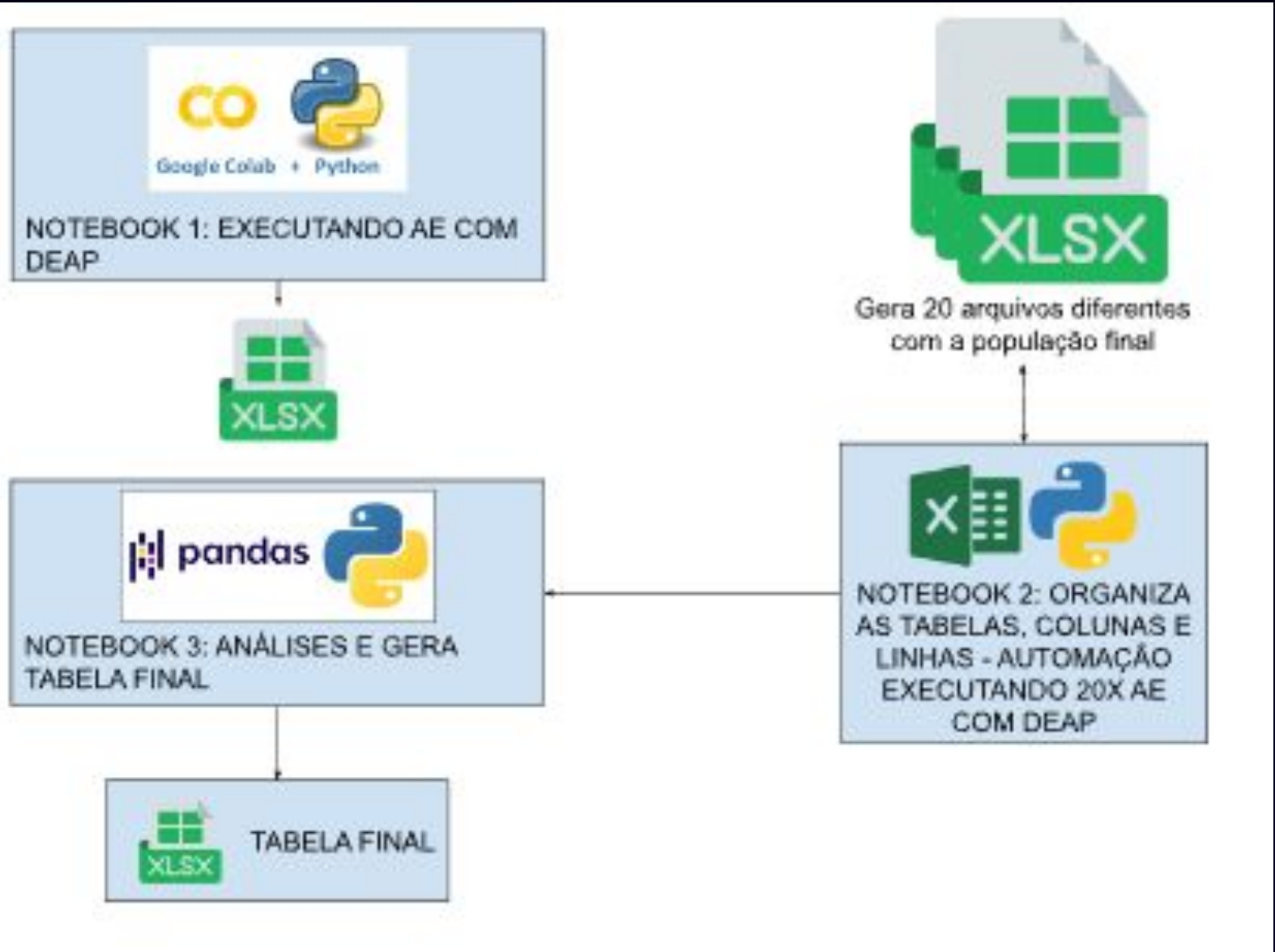
Análise dos Resultados

Os resultados das simulações são analisados e apresentados em gráficos e tabelas, demonstrando a eficácia do framework na busca por soluções ótimas, com a RCE contribuindo para a diversificação da população e a melhoria da qualidade das soluções.



Resultados e Discussões - Testes com Função de Benchmark

Automação das Simulações



Exemplos funções de otimização de Benchmarking

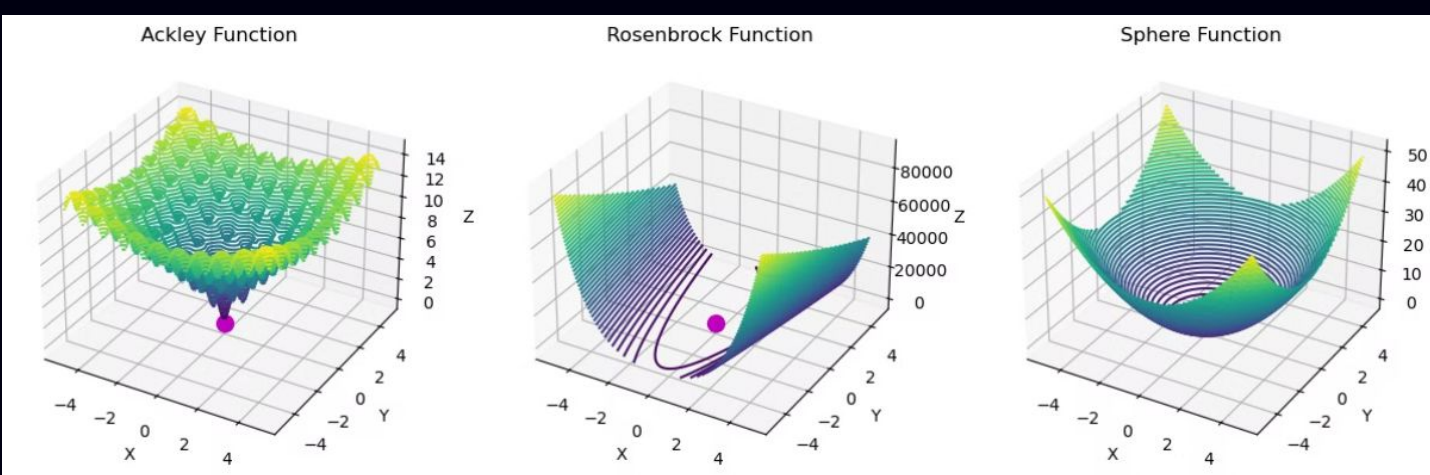
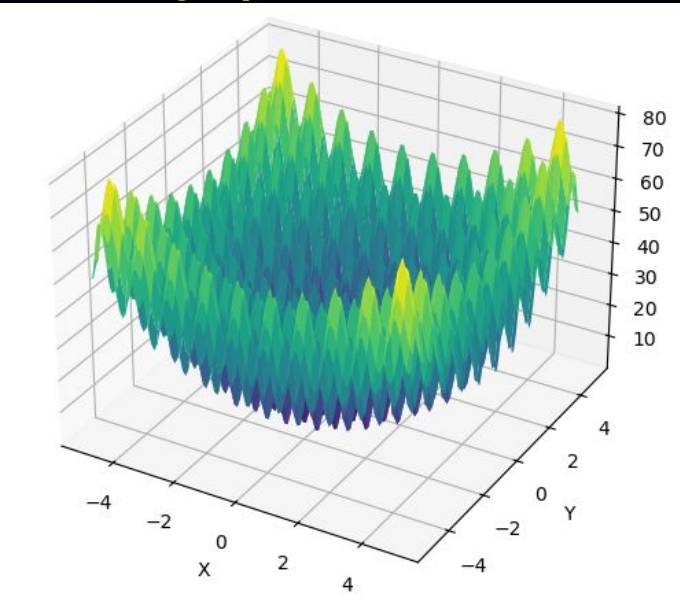


Gráfico de Benchmarking Rastrigin para duas variáveis.



$$f(\mathbf{x}) = An + \sum_{i=1}^n [x_i^2 - A \cos(2\pi x_i)]$$

Exemplo de resultados de Benchmark

1) Variando o parâmetro MUTAÇÃO

Melhor Aptidão				Número de gerações até a solução					
Melhor	Média	Desvio Padrão	CV	Variáveis Decisão	Menor Geração da Solução	Média	Desvio Padrão	CV	MUTAÇÃO
0,24	0,3	0,6	50%	[0,03; 0,018; -3,396; 3,752; 3,763; -2,554; 6,304; -4,783; -4,213; 0,735]	68	61	27,5	222%	0,05
0,03	0,41	2,68	15%	[0,006; -0,01; -4,607; 3,413; 4,448; -4,973; 2,453; 4,31; 0,294; -1,926]	57	60	23,84	252%	0,10
0,33	0,77	2,4	32%	[-0,04; -0,008; -0,785; 0,686; -0,411; 0,499; -2,357; -1,572; 1,969; -0,682]	69	68	24,18	281%	0,15
0,04	1,1	4,87	23%	[0,01; -0,011; -5,777; 3,084; -4,68; 0,010; -5,49; 0,179; -2,894]	92	64	22,26	288%	0,20

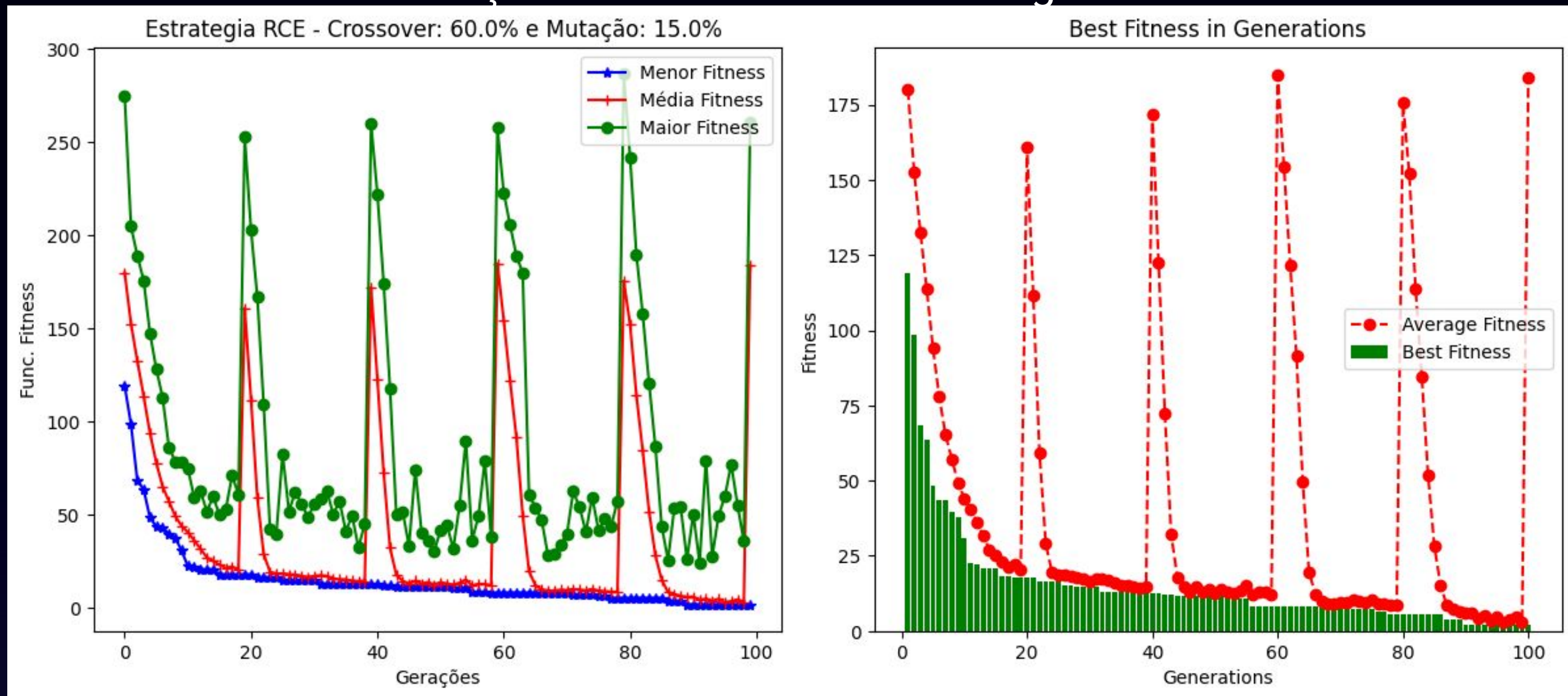
2) Variando o parametro CRUZAMENTO

Melhor Aptidão				Número de gerações até a solução					
Melhor	Média	Desvio Padrão	CV	Variáveis Decisão	Menor Geração da Solução	Média	Desvio Padrão	CV	Cruzamento
0,01	0,55	2,7	20%	[0,006; 0,003; -4,448; -0,093; -0,013; 2,961; -3,912; -3,87; -0,683; 3,969]	97	68	25,35	268%	0,20
0,08	0,16	0,83	19%	[0,02; 0,004; -3,506; 3,995; 4,28; 4,383; 5,084; 4,614; 3,493; -4,662]	31	60	21,11	284%	0,40
0,02	0,13	0,71	18%	[-0,009; 0,001; 4,015; -2,667; 1,382; 6,034; -0,479; -0,573; 3,317; 0,325]	84	60	24,45	245%	0,60
0,08	0,17	0,75	23%	[0,007; -0,018; 4,789; 1,872; 4,376; -1,471; 4,127; 2,502; -0,563; 4,007]	55	71	15,27	465%	0,90

Resultados e Discussões - Ambiente Google Colab

O framework foi implementado 100% em Python, utilizando Jupyter Notebooks para organizar o código e facilitar a execução dos testes.

Função Fitness utilizada: Rastrigin



Resultados e Discussões - População gerada pelo programa

Exemplo de amostra da população de 10 indivíduos gerada na última geração

Índice	Generations	Variáveis de Decisão	Fitness	RCE	Diversidade
0	100	[0,026; 0,026; -0,041; -0,033; -0,059; -0,003; 0,003; -0,006; 0,011; -1,016]	2,671954659	HOF	-1,092993587
1	100	[0,026; 0,026; -0,001; 0,992; 0,022; 1,038; 0,003; -0,006; 0,011; -1,016]	3,87166862	SIM	1,09759862
2	100	[0,026; 0,026; -0,001; 0,992; -0,059; 0,022; 0,003; -0,006; 0,011; -1,016]	3,19700785	SIM	-0,000614642
3	100	[0,026; 0,026; -0,041; 0,992; -0,059; -0,003; 0,003; -0,006; 0,011; -1,016]	3,445277488	SIM	-0,066925593
4	100	[0,026; 0,026; -0,041; 0,992; -1,003; -0,003; 0,003; -0,006; 0,011; -1,016]	3,758407292	SIM	-1,011124625
5	100	[0,007; 1,41; 0,782; -4,440; -4,856; -0,153; 3,211; 0,918; 4,732; -3,848]	172,2948716		-2,235588156
6	100	[-1,209; -2,266; -5,027; -1,957; -3,334; -5,081; 4,061; -3,415; -2,426; -3,512]	212,6703767		-24,16913919
7	100	[2,038; 2,297; 2,081; 1,145; 4,268; -3,190; 1,443; -0,865; -0,911; -3,427]	138,0224418		4,878388311
8	100	[4,485; -2,134; -3,269; 1,846; 4,856; 2,434; 0,778; 5,095; -2,467; 3,859]	211,0289823		15,48501801
9	100	[-1,108; -2,808; -3,256; -2,877; 0,463; 2,658; 3,3913; -4,764; 0,164; -3,936]	174,6393264		-12,07365613
10	100	[-0,751; 1,821; -2,301; -3,928; 1,015; -3,062; 2,272; 2,874; -4,205; 3,063]	128,4772309		-3,202018882

Disponibilidade e Uso do Framework



Repositório Online

O framework desenvolvido foi disponibilizado em um repositório online, juntamente com um manual de uso, permitindo que a comunidade científica possa acessar, utilizar e contribuir com o projeto.



Execução no Google Colab

Exemplos de caso de uso do framework foram disponibilizados no Google Colab, facilitando a execução e a experimentação do framework em um ambiente de computação em nuvem.

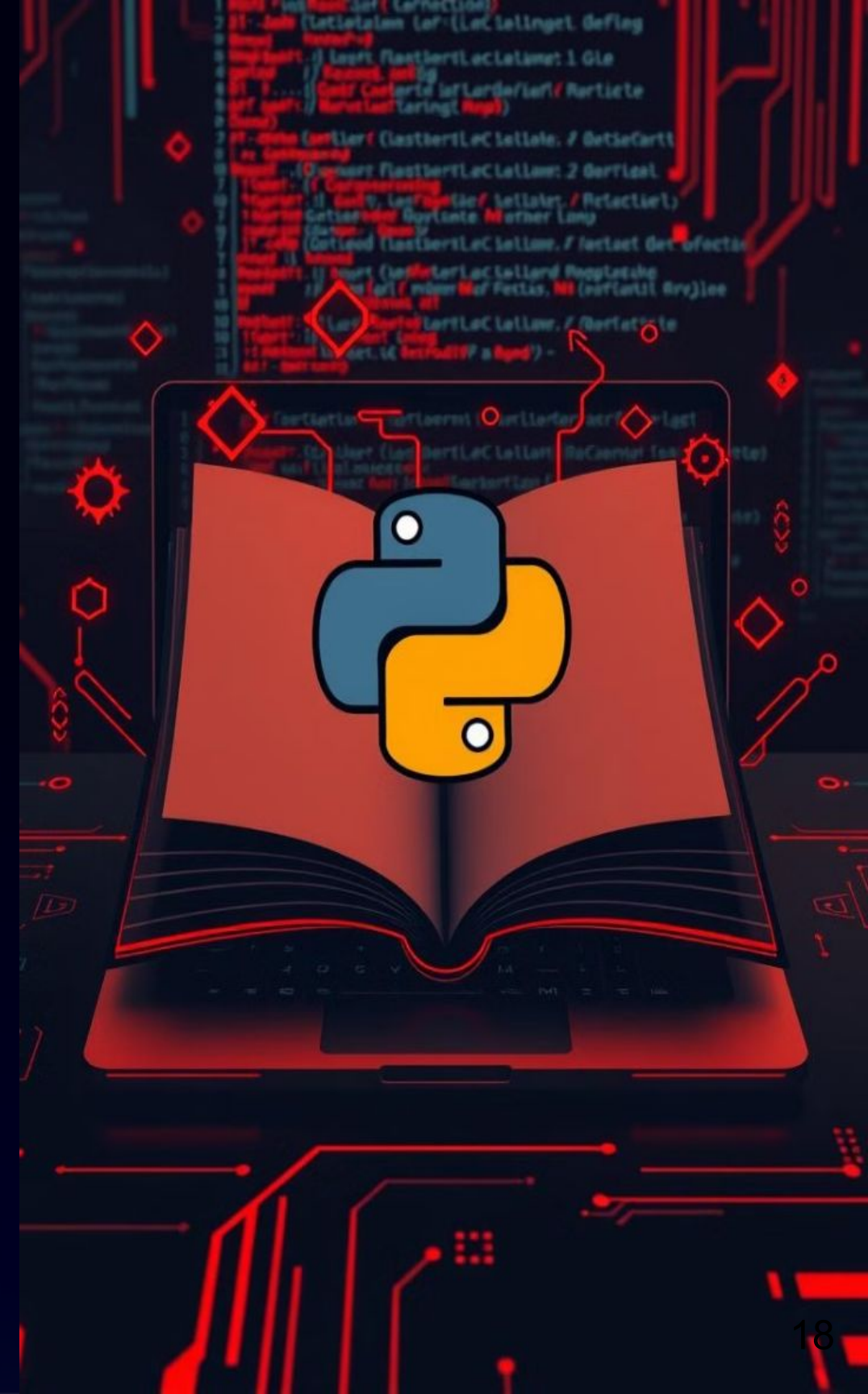


Manual de Utilização

Um manual detalhado de utilização do framework foi elaborado, descrevendo os principais recursos, a estrutura do código e as instruções para instalação e execução.

Link compartilhado:

https://drive.google.com/file/d/1KJRUA vNotNNU-80_svIVeRZ2XTo5mUlo/view?usp=sharing



Conclusões

1 Eficácia do Framework

O framework desenvolvido, utilizando Algoritmos Evolutivos e a estratégia de diversificação RCE, demonstrou-se eficaz na resolução de problemas de otimização em engenharia elétrica.

2 Diversificação e Qualidade das Soluções

A estratégia RCE contribuiu significativamente para a diversificação da população e a melhoria da qualidade das soluções encontradas.

3 Flexibilidade e Automação

O framework é open-source, flexível e modular, permitindo a adaptação a diferentes problemas de otimização e a automação de testes de benchmark.

4 Futuras Pesquisas

O trabalho abre caminho para futuras pesquisas na área de otimização de sistemas elétricos de potência, com a aplicação do framework em problemas reais e a investigação de novas estratégias de diversificação.

